



Instituto Superior de Engenharia
Politécnico de Coimbra

Departamento de Engenharia Informática e de Sistemas

Trabalho Prático

Unidade Curricular: Integração de Dados

Autores:

Maria Guimarães da Cruz Pilar Capitão - 2024129644
Thuane Vitoria Vieira de Lyra - 2024144204

Coimbra, Maio de 2026

Índice

1	Introdução	4
2	Análise de Fontes de Dados e Wrappers	5
2.1	Royal Kennel Club (JSoup)	5
2.2	Wikipédia (Expressões Regulares)	6
3	Definição do Esquema Global	8
3.1	Rigor Estrutural do Modelo Global (Análise DTD e XSD)	8
4	Gerar e Manipular Ficheiros XML (Acrescentar, Editar e Eliminar Dados)	10
5	Validação da Estrutura de Dados e Feedback ao Utilizador	12
5.1	Mecanismo de Injeção Dinâmica e Validação em Java	12
5.2	Resolução de Conflitos de Compatibilidade entre DTD e XSD	13
6	XPath	14
6.1	Pesquisas Implementadas	14
7	XSLT	16
7.1	Estratégia de Filtragem em Java para XSLT	16
7.2	Transformações Desenvolvidas em XSLT	16
8	XQuery	18
8.1	Motor de Execução XQuery	18
8.2	Queries Desenvolvidas	18
9	Interface Gráfica	20
9.1	Estrutura e Organização Visual	20
9.1.1	Painel Esquerdo — Menu de Navegação	20
9.1.2	Painel Direito — Área de Conteúdo	21
9.2	Painel Dinâmico de Pesquisas XPath	22
9.3	Separador XML Viewer	22
9.4	Separador Validações	23
9.5	Separador XSLT	24
9.6	Separador XQuery	24
9.7	Feedback ao Utilizador	25

10 Resultados e Análise	26
10.1 Cobertura do Conjunto de Dados	26
10.2 Análise por Grupo	27
10.3 Análise por Porte e Esperança de Vida	27
10.4 Testes com Raças Adicionais	27
10.4.1 Doberman	27
10.4.2 Husky Siberiano	28
10.4.3 Observações sobre os Testes Adicionais	28

Lista de Figuras

9.1	Janela principal da aplicação <i>Dog Breeds</i> — vista geral da interface. .	20
9.2	Janela XPath — vista geral da interface.	22
9.3	Janela XML — vista geral da interface.	23
9.4	Janela Validações — vista geral da interface.	23
9.5	Janela XSTL — vista geral da interface.	24
9.6	Janela XQuery — vista geral da interface.	25

Capítulo 1

Introdução

O presente relatório descreve o desenvolvimento e a implementação de uma aplicação em linguagem Java, cujo objetivo principal é a integração de dados provenientes de fontes heterogêneas, distribuídas e autónomas. O domínio do projeto foca-se na recolha e unificação de informação detalhada sobre diversas raças de cães.

Para a concretização deste sistema, foram utilizadas duas fontes web distintas: o site oficial do Royal Kennel Club e a Wikipédia . O grande desafio técnico desta integração consistiu em extrair dados não estruturados ou semiestruturados destas plataformas, efetuar a sua limpeza e transformá-los num modelo de dados unificado em formato XML. A aplicação desenvolvida permite não só a criação e validação destes ficheiros, mas também a manipulação dos dados (adição, edição e remoção), a execução de pesquisas complexas e a geração de outputs formatados em HTML e XML.

Capítulo 2

Análise de Fontes de Dados e Wrappers

Para a extração da informação requerida, foi necessário implementar wrappers específicos adaptados à natureza de cada fonte de dados, recorrendo a duas tecnologias distintas

2.1 Royal Kennel Club (JSoup)

A extração dos dados desta plataforma foi centralizada na classe `RoyalKennelWrapper`, utilizando a biblioteca `org.jsoup.Jsoup`. A abordagem técnica consistiu em aceder ao URL fornecido e instanciar um objeto `Document` utilizando o parâmetro `maxBodySize(0)`, esta decisão foi tomada devido ao facto de que as pesquisas não estavam a ser feitas a partir de certa parte do site, por isso, foi optado utilizar deste parâmetro para garantir a leitura completa. De seguida, o algoritmo utiliza seletores para iterar sobre os blocos de informação de cada cão, identificados pela classe `.m-breed-card`. Quando o nome da raça (`.mbreed-card_title`) corresponde à pesquisa efetuada, o algoritmo entra num ciclo de extração fina. Em vez de raspar texto avulso, o código navega pelos nós `.m-breed-summary_item` e avalia o texto da classe `.m-breed-summary_key-label`. Assim que encontra a chave desejada (por exemplo, "Size", "Grooming", "Sheds" ou "Town or country"), extrai e devolve o valor contido na classe adjacente (`.m-breed-summary_value`).

O wrapper lida ainda com a extração de atributos multimédia (utilizando `absUrl("src")` para garantir o caminho absoluto das fotografias) e realiza limpeza de dados no próprio momento da extração — como é visível no método `obtem_esperanca_vida`, que aplica a expressão regular `replaceAll("[^0-9]", "")` para purgar a string de caracteres alfabéticos e retornar apenas o valor numérico inteiro.

Uma particularidade do wrapper diz respeito à extração da **tagline** de cada raça — uma frase descritiva curta que o Royal Kennel Club apresenta no cabeçalho da página individual de cada cão. Ao contrário de todos os outros campos, que são extraídos da página de listagem geral (*breeds-a-to-z*), a tagline exige **dois pedidos HTTP sequenciais**: o método `obtem_link_raca` percorre os cartões da listagem com o seletor `a.m-breed-card__link` e, ao encontrar o cartão cujo título corresponde à raça pretendida, devolve o URL absoluto da página individual dessa

raça (via `absUrl("href")`). De seguida, o método `obtem_tagline` acede a esse URL e extrai o texto do elemento com a classe `.o-breed-header__tagline`, devolvendo "Sem tagline." caso o elemento não exista. Esta separação em dois métodos tem ainda uma função adicional: `obtem_link_raca` é usado como **mecanismo de verificação** antes de iniciar qualquer extração — se devolver `null`, a raça não existe no Royal Kennel Club e o processo é abortado de imediato.

Do ponto de vista da integração de dados, a tagline **não é persistida nos ficheiros XML**. Em vez disso, é obtida dinamicamente pela interface gráfica no momento em que o utilizador pesquisa uma raça: o `MainFrame` invoca `obtem_tagline` e inclui o texto resultante no bloco de resultados apresentado na `JTextArea`, entre aspas, a seguir ao nome da raça. Esta decisão de design evita guardar dados textuais de apresentação na base de dados estruturada, mantendo os ficheiros XML focados em informação técnica e morfológica.

2.2 Wikipédia (Expressões Regulares)

A segunda fonte de dados utilizada foi a Wikipédia em inglês, acedida através de pedidos HTTP diretos realizados pela classe `HttpRequestFunctions`, que guarda o HTML da página de cada raça num ficheiro local temporário (`raca.html`). A extração dos campos desejados foi inteiramente implementada na classe `WikiWrapperER`, recorrendo **exclusivamente a Expressões Regulares** (pacote `java.util.regex`).

A principal dificuldade desta fonte reside no facto de o HTML da Wikipédia ser gerado dinamicamente por *templates* de infobox que variam consoante a raça. Para contornar essa heterogeneidade, foi adotada uma técnica de *acumulação de linhas*: o algoritmo concatena sucessivamente as linhas do ficheiro numa única cadeia de caracteres em crescimento contínuo, e aplica o `Matcher` a cada iteração. Esta abordagem permite capturar padrões que se distribuem por múltiplas linhas do HTML sem necessitar de pré-processamento adicional.

Foram implementados os seguintes métodos de extração:

- **obtem_origem**: Aplica duas expressões regulares de forma complementar. A primária (`Origin</th>.*?<[>]*>(<[<]+<)`) extrai o país da infobox estruturada. Caso o campo não exista (páginas sem infobox), uma ER alternativa (`originated in ([A-Z][a-z]+)`) procura a menção do país em linguagem natural no corpo do artigo, garantindo uma cobertura mais ampla.
- **obtem_pesoMacho / obtem_pesoFemea**: Utilizam expressões regulares de correspondência *greedy* que percorrem as células da infobox à procura das palavras-chave `Weight` e `Males/Females`, extraindo de seguida o texto do elemento `<td>` ou `<li>` adjacente. Quando não é encontrada distinção de sexo, ambos os métodos devolvem a *String* "Sem distinção de sexo".
- **obtem_pesoGeral**: Trata os casos em que a Wikipédia apresenta um único valor de peso sem distinção de sexo. A ER apanha diretamente o conteúdo da célula adjacente ao cabeçalho `Weight`. No `DogBreedsApp`, a lógica de integração determina que este valor só é usado quando tanto `parseMedida(pesoMacho)`

como `parseMedida(pesoFemea)` retornam `null`, garantindo que o nó `<geral>` e os nós `<macho>`/`<femea>` nunca coexistem no XML.

- **obtem_cores:** Extrai as cores da pelagem da infobox, usando a ER `Colou?r</th>` (com o `u?` para cobrir tanto a grafia britânica *Colour* como a americana *Color*), navegando pelos elementos filhos da célula para atingir o texto visível. A linha é adicionalmente limpa de blocos `<style>` embutidos antes de aplicar o `Matcher`.
- **obtem_alturaMacho / obtem_alturaFemea / obtem_alturaGeral:** Seguem a mesma lógica de distinção de sexo aplicada ao peso, mas navegam pela estrutura de tabela de forma sequencial: a ER para o macho localiza a primeira linha após o cabeçalho `Height`, a ER para a fêmea avança para a linha `<tr>` seguinte evitando capturas cruzadas, e o método geral captura diretamente o `<td>` adjacente ao cabeçalho quando não existe separação por sexo.

Uma vez extraídos, todos os valores métricos são processados pelo método utilitário **parseMedida**. Este método normaliza o texto antes de aplicar as expressões regulares, substituindo variantes tipográficas (traço longo `-`, espaço não-separável ` `), sinónimos em inglês (*kilograms*, *centimetres*, *pounds*, *inches*) e a expressão *"X to Y"* pela notação de intervalo *"X-Y"*. Seguidamente, aplica uma sequência de padrões por ordem decrescente de especificidade: primeiro tenta capturar intervalos já expressos em unidades métricas (**kg/cm**); caso falhe, tenta capturar valores em unidades imperiais (**lb/in**) e invoca o método **converteParaMetrico**, que aplica os fatores de conversão 0.453592 (libras para kg) e 2.54 (polegadas para cm), arredondando a uma casa decimal. Este pipeline garante que os dados de peso e altura são sempre armazenados em unidades SI, independentemente da convenção utilizada na página da Wikipédia.

Capítulo 3

Definição do Esquema Global

O modelo global de integração foi arquitetado com base em dois ficheiros XML relacionais, visando otimizar a organização e a pesquisa da informação:

- `racas.xml`: Funciona como um índice ou catálogo principal. Contém apenas a listagem das raças, armazenando o nome da raça e atribuindo-lhe um identificador único (ID).
- `detalhes.xml`: Atua como a base de dados profunda do sistema. Todos os atributos técnicos e morfológicos extraídos pelos wrappers (tamanho, peso, esperança de vida, etc.) são guardados aqui, associados ao ID gerado no ficheiro principal.

A estrutura hierárquica do `detalhes.xml` foi desenhada para maximizar a granularidade da informação. Em vez de aglomerar dados numa única tag (por exemplo, um intervalo de peso), optou-se por subdividir os elementos estruturalmente, criando nós de género como `<macho>`, `<femea>` e `<geral>` contidos dentro das respectivas categorias de `<peso>` e `<altura>`. Esta decisão arquitetural facilitou significativamente o isolamento de variáveis e a precisão das pesquisas e transformações.

3.1 Rigor Estrutural do Modelo Global (Análise DTD e XSD)

A modelação e estruturação dos dados foram projetadas para refletir a semântica complexa do domínio das raças caninas, tirando partido de mecanismos avançados de modelação XML. Enquanto o par de ficheiros `racas` funciona como um índice relacional plano (onde cada elemento `<breed>` tem obrigatoriamente um ID e um atributo nome definido como requerido), a base de dados profunda presente em `detalhes` recorre a uma modelação altamente granularizada.

No ficheiro `detalhesXSD.xsd`, evitámos a utilização de nós de texto genéricos para dados quantitativos. Em vez disso, definimos estruturas de dados reutilizáveis através de tipos complexos (Complex Types):

- `medidaType`: Modela uma especificação métrica exata, obrigando à existência coordenada de três elementos filhos: `<min>` (valor mínimo), `<max>` (valor máximo) e `<unidade>` (unidade de medida, ex: cm ou kg).

- `dimensaoType`: Modela as vertentes biológicas de medição, permitindo agrupar de forma opcional (`minOccurs="0"`) os elementos `<macho>`, `<femea>` ou `<geral>`, cada um deles rigidamente associado ao tipo `medidaType`.

Esta abordagem garante que o elemento `<tamanhoDetalhes>` possua uma árvore estritamente tipada e homogênea. Se a fonte de dados (Wikipédia) fornecer dados de peso sem distinção de sexo, o XSD valida a inserção do nó `<geral>` isolado. Caso distinga os géneros, valida a inserção paralela de `<macho>` e `<femea>`. Este nível de rigor na modelagem eliminou a necessidade de efetuar rotinas secundárias de limpeza de strings em Java durante as operações de pesquisa avançada por XPath.

Capítulo 4

Gerar e Manipular Ficheiros XML (Acrescentar, Editar e Eliminar Dados)

A manipulação dos dados nos ficheiros XML foram implementadas recorrendo à API JDOM2, garantindo uma gestão eficaz da árvore documental (DOM). A lógica de manipulação foi isolada na classe ModeloXML, separando o motor de dados da interface gráfica.

- **Adicionar Dados:** A inserção de novas raças é despoletada pela interface, que invoca o wrapper para extrair a informação da web e mapeá-la para objetos Java (Breed e BreedDetails). Na classe ModeloXML, os métodos adicionaRaca e adicionaDetalhes verificam se os documentos já existem; caso contrário, instanciam um novo Document e o respetivo elemento raiz (<racas> ou <detalhes>). A construção da árvore é feita através da criação de objetos Element e Attribute. Destaca-se o tratamento de objetos complexos aninhados, como o SizeInfo, onde o código constrói proativamente a hierarquia XML injetando os elementos <macho> e <femea> dentro de <peso>, e este por sua vez dentro de <tamanhoDetalhes>.
- **Remover Dados:** A operação de eliminação foi desenhada para garantir a integridade referencial entre os dois ficheiros. O método removeRaca inicia a iteração no ficheiro racas.xml, procurando o elemento <breed> cujo atributo nome corresponda à pesquisa do utilizador. Ao encontrar uma correspondência, o algoritmo guarda temporariamente o <id> da raça e utiliza o comando `esc.getParent().removeContent(esc)` para apagar o nó por completo. De seguida, o método removeDetalhes utiliza o ID previamente guardado para localizar e eliminar o bloco <breedDetail> correspondente no ficheiro detalhes.xml. No final, as alterações a ambos os documentos são consolidadas no disco.
- **Editar Dados:** Para a edição pontual de atributos, a interface recorre a caixas de diálogo interativas (JOptionPane) para solicitar ao utilizador o nome da raça, o campo a editar e o novo valor. A nível de processamento lógico, a aplicação acede primeiramente ao racas.xml para mapear o nome introduzido para o seu respetivo ID. De seguida, o fluxo é encaminhado através de uma

estrutura switch para métodos altamente específicos na classe ModeloXML (tais como alteraGrupo, alteraCores, alteraEsperancaVida, etc.). Estes métodos iteram sobre os nós filhos de detalhes.xml, localizam o breedId correto e aplicam o método .setText(novoValor) à tag específica, sobrescrevendo a informação anterior sem afetar o resto da estrutura do documento.

Capítulo 5

Validação da Estrutura de Dados e Feedback ao Utilizador

Para assegurar a integridade semântica e a robustez da base de dados local ao longo do ciclo de vida da aplicação (especialmente após sucessivas operações de adição, remoção e edição de dados pela interface), foi implementado um mecanismo de validação estrito baseado em esquemas DTD (Document Type Definition) e XSD (XML Schema Definition).

Em vez de adotar uma abordagem estática (que obrigaria a introduzir manualmente as referências de validação no cabeçalho dos ficheiros físicos XML), optou-se por uma arquitetura de injeção dinâmica em tempo de execução, centralizada na classe autónoma `validarXML`.

5.1 Mecanismo de Injeção Dinâmica e Validação em Java

O processo de validação dinâmica divide-se nos seguintes passos práticos:

1. **Leitura:** O ficheiro XML alvo é carregado para a memória do programa.
2. **Injeção de Regras:** O sistema deteta o tipo de validação escolhido e associa as regras correspondentes (o ficheiro DTD ou XSD) diretamente à raiz do documento. Isto evita que as referências de validação tenham de estar declaradas de forma estática nos ficheiros originais.
3. **Análise e Validação:** O documento temporário modificado é testado pelo motor interno do Java, que avalia se a estrutura e a semântica dos dados cumprem todas as regras exigidas.
4. **Nota de Arquitetura (Tratamento de Retornos):** Para garantir o desacoplamento, a classe `validarXML` não emite caixas de diálogo. Ela processa a validação e retorna códigos inteiros estritos: 1 para um documento perfeitamente válido, -1 para erros ou falhas semânticas na estrutura, e 0 caso os ficheiros físicos ou os esquemas estejam em falta no diretório do projeto.

No lado da interface gráfica (MainFrame), os métodos associados aos eventos dos botões (como `jButtonValidarRacasdtdActionPerformed` ou `jButtonValidarDetalhesxsdActionPerformed`) capturam estes códigos numéricos e encarregam-se de traduzi-los numa experiência visual clara para o utilizador, recorrendo a componentes `JOptionPane` para exibir mensagens interativas personalizadas de sucesso, erro de estrutura ou aviso técnico.

5.2 Resolução de Conflitos de Compatibilidade entre DTD e XSD

Durante os testes integrados de validação, foi detetado um conflito técnico crítico devido à execução consecutiva de validações XSD e DTD sobre os mesmos ficheiros. Quando a validação XSD é executada, a aplicação injeta os atributos de namespace `xmlns:xsi` e `xsi:noNamespaceSchemaLocation` no nó raiz do XML. Se o utilizador invocar a validação por DTD imediatamente a seguir, o parser SAX lança uma exceção de infraestrutura, reportando que esses atributos específicos do XSD não se encontram declarados no documento DTD. Para contornar esta limitação e assegurar a total interoperabilidade da aplicação, foi desenhada uma solução cirúrgica a nível dos ficheiros de definição. Acrescentámos no final de ambos os ficheiros DTD (`racasDTD.dtd` e `detalhesDTD.dtd`) uma diretiva `<!ATTLIST>` explícita para o elemento de topo:

```
<!ATTLIST racas
  xmlns:xsi CDATA #IMPLIED
  xsi:noNamespaceSchemaLocation CDATA #IMPLIED
>
```

Esta instrução define os atributos introduzidos pelo XSD como implícitos (`#IMPLIED`), forçando o motor de parsing do DTD a tolerar e validar com sucesso a presença dessas propriedades adicionais. Desta forma, o sistema ganha a flexibilidade de alternar livremente entre ambos os tipos de validação em tempo de execução sem corromper a leitura dos dados.

Capítulo 6

XPath

A funcionalidade de pesquisa avançada sobre a base de dados local foi implementada recorrendo à linguagem XPath, integrada na aplicação através da API de XPath da biblioteca JDOM2 (`org.jdom2.xpath`). Toda a lógica de pesquisa foi centralizada na classe `XPathSearches`, mantendo o princípio de separação de responsabilidades: esta classe recebe os documentos XML já carregados em memória (passados como argumentos), constrói e avalia as expressões XPath, e devolve os resultados formatados como texto para apresentação na interface.

6.1 Pesquisas Implementadas

Foram implementadas doze pesquisas distintas, que cobrem um espectro alargado de critérios de filtragem:

- **Por Nome** (`pesquisarPorNome`): Realiza primeiro uma consulta ao `racas.xml` para mapear o nome introduzido pelo utilizador ao respetivo `breedId`, e só depois constrói a expressão `//breedDetail[breedId='X']` para pesquisar no ficheiro de detalhes. Esta indireção é necessária porque o nome da raça apenas existe no ficheiro de índice.
- **Por Porte** (`pesquisarPorPorte`): Pesquisa exata sobre o elemento `<porte>` com a expressão `//breedDetail[porte='X']`.
- **Por Peso** (`pesquisarPorPeso`): Aplica um predicado numérico sobre todos os nós filhos do elemento `<peso>` (macho, fêmea ou geral) com a expressão `//breedDetail[tamanhoDetalhes/peso/*[number(min) >= A and number(max) <= B]]`, utilizando a função `number()` para forçar a conversão dos valores de texto para número.
- **Por Característica** (`pesquisarPorCaracteristica`): Pesquisa genérica por qualquer campo de texto, aplicando a função `contains()` combinada com `translate()` para normalização de maiúsculas/minúsculas, tornando a pesquisa insensível ao caso.
- **Por Origem com Porte Opcional** (`pesquisarPorOrigem`): Constrói dinamicamente a expressão XPath: se o utilizador fornecer também um porte, o

predicado é expandido com uma condição **and** adicional; caso contrário, filtra apenas por país de origem.

- **Por Cor** (`pesquisarPorCor`): Utiliza `contains()` insensível ao caso sobre o elemento `<cores>`.
- **Por Grupo** (`pesquisarPorGrupo`): Análogo à pesquisa por cor, aplicado ao elemento `<grupo>`.
- **Por Esperança de Vida Mínima** (`pesquisarPorEsperancaVida`): Seleciona todas as raças cuja esperança de vida é maior ou igual ao limiar introduzido, com `//breedDetail[number(esperancaVida) >= N]`.
- **Maior Esperança de Vida** (`pesquisarMaiorEsperancaVida`): Implementa o clássico padrão XPath de máximo sem funções de agregação, seleciona os nós para os quais não existe nenhum outro nó com valor superior, retornando o(s) registo(s) com a maior esperança de vida.
- **Raças Sem Dados de Peso** (`pesquisarSemDadosPeso`): Utiliza o predicado de negação `not(tamanhoDetalhes/peso/*)` para identificar raças para as quais a Wikipédia não forneceu dados de peso, expondo lacunas na integração de dados.
- **Por Altura** (`pesquisarPorAltura`): Análoga à pesquisa por peso, aplicada ao elemento `<altura>` com um intervalo em centímetros.
- **Por 3 Critérios Combinados** (`pesquisarCombinado3`): Combina num único predicado XPath três condições simultâneas sobre `tamanhoPelo`, `nivelExercicio` e `paisOrigem`, todas com normalização de maiúsculas/minúsculas via `translate()`, demonstrando o poder expressivo do XPath na filtragem multi-critério.

Capítulo 7

XSLT

7.1 Estratégia de Filtragem em Java para XSLT

O método utilizado para processar os documentos (`transformaDocumento2` da classe `JDOMFunctions_XSLT`) apresenta uma limitação prática: ele lê o ficheiro XML de forma fechada. Isto significa que não permite passar variáveis externas (como o nome da raça que o utilizador acabou de pesquisar) diretamente do código Java para dentro do ficheiro `.xsl`.

Para contornar esta restrição de forma inteligente, foi utilizada uma estratégia de "clonagem de nós em memória":

Quando o utilizador pesquisa por uma raça, o Java mapeia esse nome para o respetivo ID consultando o `racas.xml`. De seguida, carrega o ficheiro `detalhes.xml` completo para a memória. Para isolar apenas o cão pretendido e não corromper o documento original, o algoritmo cria um novo documento XML temporário e vazio na memória.

Através de um ciclo de iteração clássico, a aplicação procura o nó do cão correto e utiliza o método `.clone()` da API `JDOM2` para copiar exclusivamente esse elemento e todos os seus filhos para o novo documento temporário. O documento XML resultante, contendo agora apenas os dados isolados da raça pesquisada, é finalmente guardado e enviado de forma perfeitamente limpa para o motor XSLT.

7.2 Transformações Desenvolvidas em XSLT

Tirando partido de propriedades e eixos de navegação avançados da linguagem de transformações, foram materializados os seguintes outputs:

- **Catálogo Geral Fotográfico (HTML):** Esta transformação percorre os detalhes para gerar uma tabela visual interativa. Como os nomes textuais das raças residem exclusivamente noutro ficheiro, utilizámos a função nativa do XSLT `document('racas.xml')` combinada com a variável local `$idRaca`. Esta técnica permite ao XSLT abrir um segundo ficheiro, cruzar o ID e extrair o nome.
- **Listagem XML por Porte:** Esta transformação gera um documento XML organizado por categorias de porte (*Small*, *Medium*, *Large*). O ficheiro XSL

percorre os três valores de porte possíveis e, para cada um, seleciona com um predicado XPath todas as raças cujo elemento `<porte>` corresponde ao valor em curso. O cruzamento com o `racas.xml` é feito através da função `document()`, permitindo incluir o nome de cada raça no output. O resultado é um ficheiro XML limpo, agrupado e ordenado que pode ser reutilizado como input para outras transformações ou análises.

- **Ficha Técnica Simplificada (HTML):** Desenhada para extrair dados técnicos limpos sem multimédia. Destaca-se pelo uso de blocos condicionais (`<xsl:if test="...">`), assegurando que as secções métricas de peso e altura só são renderizadas na estrutura do HTML se essas tags existirem fisicamente na árvore do cão selecionado.
- **Exportação de Dados Brutos (XML):** Permite extrair o registo individual de uma raça. O desafio desta operação consistiu no comando tradicional `<xsl:copy-of>`, que herdava e imprimia os atributos de validação do XSD na nova árvore, gerando poluição visual. A solução implementada baseou-se num molde recursivo usando `<xsl:element name="{local-name()}">`, que extrai estritamente o nome da tag, recriando uma estrutura XML perfeitamente purificada de namespaces.

No fim de cada geração HTML, a interface invoca a classe `java.awt.Desktop` para apresentar automaticamente o resultado no navegador padrão do sistema.

Capítulo 8

XQuery

Para a geração de relatórios analíticos e estatísticos sobre a base de dados de raças, foi adotada a linguagem XQuery, executada através do motor **Saxon HE** (Home Edition). A integração com o código Java foi implementada na classe `SaxonFunctions_XQuery`, que utiliza a API clássica do Saxon (`net.sf.saxon.query`) para compilar e executar as queries armazenadas em ficheiros `.xql` externos.

8.1 Motor de Execução XQuery

A classe `SaxonFunctions_XQuery` expõe três métodos estáticos — `xQueryToText`, `xQueryToHtml` e `xQueryToXml` — que diferem apenas no método de serialização do resultado. Todos seguem a mesma sequência de execução: instanciam um objeto `Configuration Saxon`, criam um `StaticQueryContext` com o URI base do diretório de trabalho (`user.dir`), compilam a query lida do ficheiro `.xql` via `compileQuery(new FileReader(queryFile))`, criam um `DynamicQueryContext` para a avaliação em tempo de execução, e invocam `exp.run()` que escreve o resultado diretamente num `StreamResult` apontado para o ficheiro de output.

Na interface gráfica, os três botões do separador XQuery invocam `SaxonFunctions_XQuery.xQueryToText(outputFile, queryFile)`, onde `outputFile` é o ficheiro `.txt` de destino e `queryFile` é o ficheiro `.xql` com a query. Após a execução, o conteúdo do ficheiro de output é lido e apresentado na `JTextArea` do separador.

A escolha por ficheiros `.xql` externos em detrimento de queries embutidas no código Java foi deliberada: permite modificar as análises sem recompilar a aplicação e mantém o código Java desacoplado da lógica de consulta.

8.2 Queries Desenvolvidas

Foram desenvolvidas três queries XQuery, cada uma orientada a um tipo de análise distinto:

- **Ranking por Esperança de Vida** (`ranking_vida.xql`): Esta query cruza os dois ficheiros XML — abre o `detalhes.xml` para obter os valores numéricos de esperança de vida e consulta dinamicamente o `racas.xml` para recuperar o nome de cada raça através do seu `breedId`. Os resultados são ordenados

de forma decrescente pela esperança de vida com a cláusula `order by ... descending`, produzindo um ranking legível onde cada entrada inclui o nome, a longevidade, o porte e o grupo da raça.

- **Raças por Grupo** (`grupos_racas.xql`): Realiza um agrupamento dinâmico das raças por categoria (grupo), utilizando `distinct-values()` para extrair os grupos únicos e um ciclo `FLWOR` aninhado para listar, por ordem alfabética, todas as raças pertencentes a cada grupo. O cruzamento com `racas.xml` é feito em tempo de execução, demonstrando a capacidade de *join* implícito do XQuery entre documentos.
- **Estatísticas Gerais** (`estatisticas.xql`): Produz um relatório sumário da base de dados, calculando o total de raças registadas, a média, o máximo e o mínimo da esperança de vida através das funções de agregação nativas `count()`, `avg()`, `max()` e `min()`. Adicionalmente, gera distribuições analíticas por porte, por grupo e por país de origem, utilizando `distinct-values()` combinado com `count()` e ordenação alfabética.

Os resultados de cada query são persistidos em ficheiros `.txt` no diretório do projeto (`ranking_vida.txt`, `grupos_racas.txt`, `estatisticas.txt`) e apresentados simultaneamente na interface gráfica.

Capítulo 9

Interface Gráfica

A interface gráfica da aplicação foi desenvolvida em **Java Swing**, utilizando o ambiente de design visual do NetBeans IDE (ficheiro `.form`). O componente central é a classe `MainFrame`, que estende `javax.swing.JFrame` e integra todos os módulos funcionais da aplicação numa única janela. A Figura 9.1 apresenta o aspeto geral da interface.

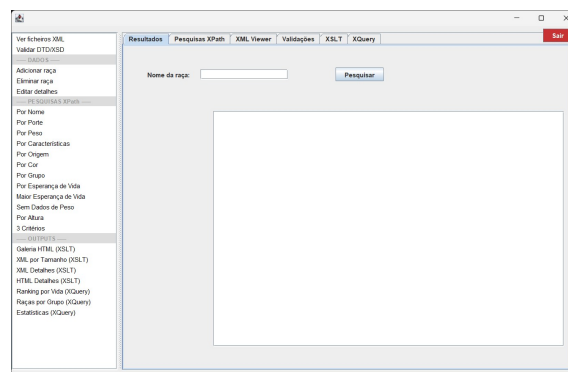


Figura 9.1: Janela principal da aplicação *Dog Breeds* — vista geral da interface.

9.1 Estrutura e Organização Visual

A janela principal assenta sobre um `JSplitPane` com divisão vertical, que separa a área de navegação à esquerda da área de conteúdo à direita. No canto superior direito existe ainda um botão **Sair**, com fundo vermelho, que solicita confirmação ao utilizador antes de encerrar a aplicação, através de um `JOptionPane.showConfirmDialog`.

9.1.1 Painel Esquerdo — Menu de Navegação

O painel esquerdo contém um `JList` com todos os módulos da aplicação, organizados em quatro secções claramente delimitadas por cabeçalhos visuais:

- **(topo):** Ver ficheiros XML, Validar DTD/XSD.
- **DADOS:** Adicionar raça, Eliminar raça, Editar detalhes.

- **PESQUISAS XPath:** doze opções de pesquisa — Por Nome, Por Porte, Por Peso, Por Características, Por Origem, Por Cor, Por Grupo, Por Esperança de Vida, Maior Esperança de Vida, Sem Dados de Peso, Por Altura e 3 Critérios combinados.
- **OUTPUTS:** sete opções de geração de output — Galeria HTML (XSLT), XML por Tamanho (XSLT), XML Detalhes (XSLT), HTML Detalhes (XSLT), Ranking por Vida (XQuery), Raças por Grupo (XQuery) e Estatísticas (XQuery).

Para distinguir visualmente os cabeçalhos de secção dos itens clicáveis, foi implementado um `DefaultListCellRenderer` personalizado. Os itens identificados como cabeçalhos — cujo texto começa pela sequência "--" — são renderizados com fundo cinzento-escuro, texto a branco e fonte em negrito, tornando-os visualmente distintos e não seleccionáveis pelo utilizador.

A seleção de um item no `JList` despoleta o método `tratarSelecaoMenu`, que interpreta o índice selecionado e executa a ação correspondente: navega para o separador adequado, mostra o cartão de pesquisa correto, ou invoca diretamente a lógica de negócio associada.

9.1.2 Painel Direito — Área de Conteúdo

O painel direito é composto por um `JTabbedPane` com seis separadores, cada um dedicado a um domínio funcional distinto:

- **Resultados:** apresenta os resultados das pesquisas XPath e das operações de dados. Inclui um campo de texto "Nome da raça:" com um botão "Pesquisar" para pesquisa direta por nome, e uma `JTextArea` de grande dimensão para exibição dos resultados formatados.
- **Pesquisas XPath:** aloja o painel dinâmico de formulários de pesquisa, implementado com `CardLayout` (descrito na secção seguinte).
- **XML Viewer:** permite visualizar o conteúdo em bruto dos ficheiros XML (`racas.xml` e `detalhes.xml`) numa área de texto com formatação monoespaçada.
- **Validações:** apresenta os resultados das validações DTD e XSD, indicando se os ficheiros XML estão conformes com os esquemas definidos, com mensagens de sucesso ou de erro detalhadas.
- **XSLT:** agrupa os botões de geração das quatro transformações XSLT disponíveis. Após a transformação, o resultado textual é apresentado numa `JTextArea`; no caso de outputs HTML, a interface abre automaticamente o ficheiro no navegador padrão através de `java.awt.Desktop.getDesktop().browse()`.
- **XQuery:** agrupa os três botões de execução das queries XQuery (Ranking por Vida, Raças por Grupo, Estatísticas Gerais). Os resultados são apresentados numa `JTextArea` e guardados em ficheiros `.txt` no diretório do projeto.

9.2 Painel Dinâmico de Pesquisas XPath

Uma das decisões de design mais relevantes foi a adoção de um `CardLayout` no separador de Pesquisas XPath. Dado que cada um dos doze tipos de pesquisa requer um conjunto diferente de campos de entrada — alguns pedem apenas uma *String*, outros pedem dois valores numéricos, e outros combinam múltiplos campos opcionais, o `CardLayout` permite apresentar ao utilizador apenas os campos relevantes para a pesquisa selecionada, sem sobrecarregar visualmente a interface.

Quando o utilizador seleciona uma das doze opções de pesquisa XPath no `JList`, o método `tratarSelecaoMenu` calcula o índice do cartão correspondente e invoca `xpathCardLayout.show()` para apresentar o painel correto, atualizando simultaneamente o `JLabel` de título com a descrição da pesquisa ativa (por exemplo, "*Pesquisa: Por Peso (kg)*"). Os resultados são sempre enviados para **Resultados**.

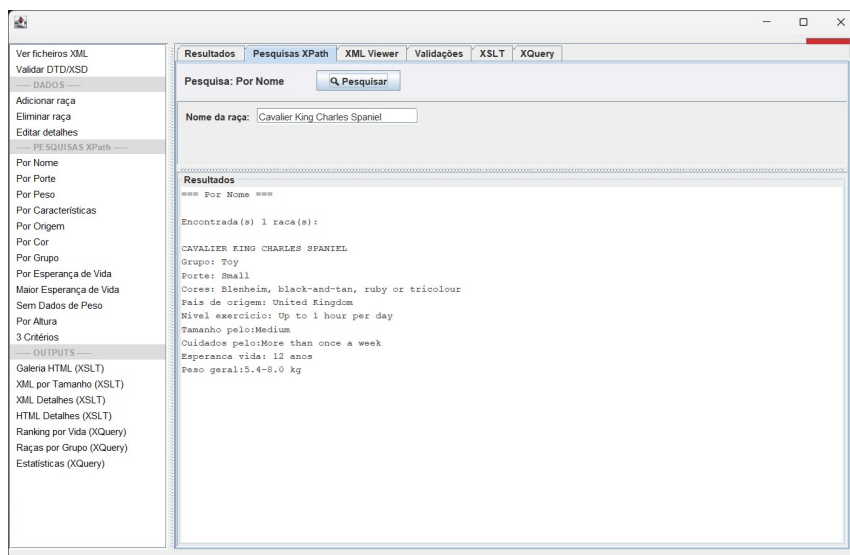


Figura 9.2: Janela XPath — vista geral da interface.

9.3 Separador XML Viewer

O separador **XML Viewer** permite inspecionar o conteúdo em bruto dos ficheiros XML que constituem a base de dados local da aplicação — `racas.xml` e `detalhes.xml`. O conteúdo é carregado e apresentado numa `JTextArea` com formatação monoespaciada, facilitando a leitura da estrutura hierárquica do XML. Esta funcionalidade é invocada através da opção "*Ver ficheiros XML*" no menu lateral, e é útil para verificar rapidamente o estado atual da base de dados após operações de adição, edição ou remoção de raças.

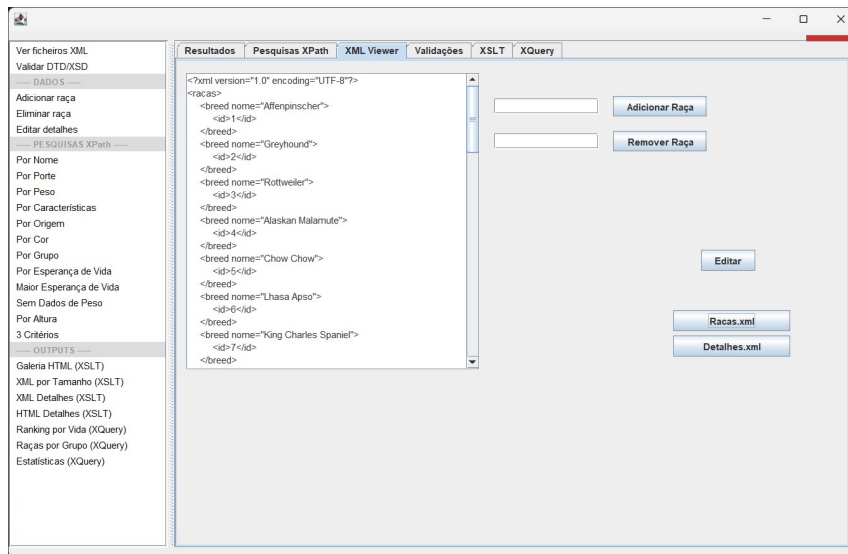


Figura 9.3: Janela XML — vista geral da interface.

9.4 Separador Validações

O separador **Validações** centraliza os resultados das verificações de conformidade dos ficheiros XML. A funcionalidade é ativada através da opção "*Validar DTD/XSD*" no menu lateral, que despoleta a validação dos ficheiros `racas.xml` e `detalhes.xml` contra os respetivos esquemas. A validação DTD é injetada dinamicamente nos documentos em memória antes de ser avaliada; a validação XSD recorre à API `javax.xml.validation.Validator`. O resultado — sucesso ou lista de erros de conformidade — é apresentado numa `JTextArea` dedicada, com mensagens descritivas que identificam o tipo de problema e a localização no documento.

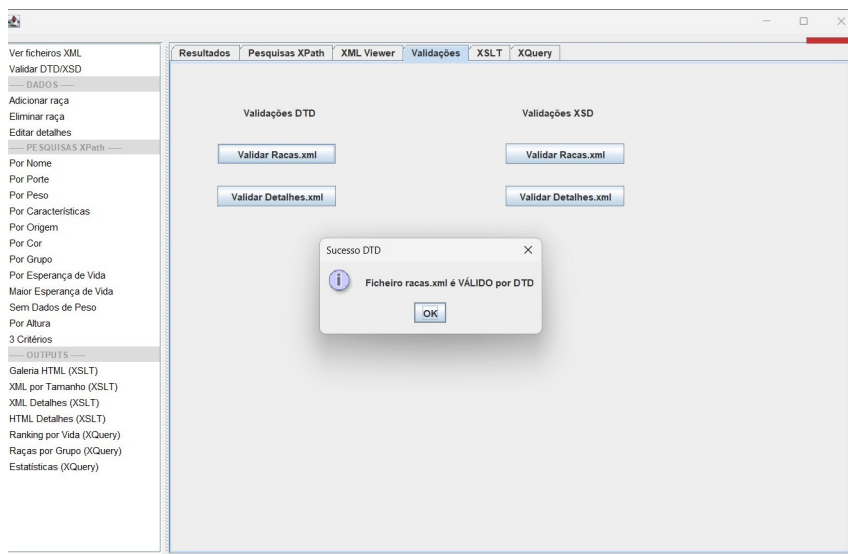


Figura 9.4: Janela Validações — vista geral da interface.

9.5 Separador XSLT

O separador **XSLT** agrupa os quatro botões de geração de outputs por transformação XSLT: *Galeria HTML*, *XML por Tamanho*, *XML Detalhes* e *HTML Detalhes*. Cada botão invoca o método `JDOMFunctions_XSLT.transformaDocumento2()`, que aplica a folha de estilos `.xsl` correspondente ao documento XML em memória e serializa o resultado. O conteúdo transformado é apresentado na `JTextArea` do separador; nos casos em que o output é um ficheiro HTML, a interface invoca adicionalmente `java.awt.Desktop.getDesktop().browse()` para abrir o resultado diretamente no navegador padrão do sistema, proporcionando uma visualização imediata ao utilizador.

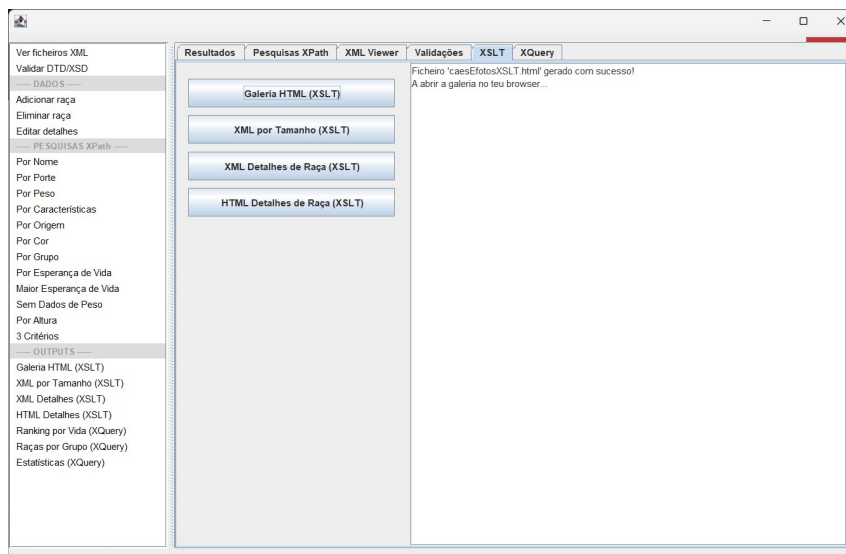


Figura 9.5: Janela XSTL — vista geral da interface.

9.6 Separador XQuery

O separador **XQuery** disponibiliza três botões correspondentes às queries implementadas: *Ranking por Vida*, *Raças por Grupo* e *Estatísticas Gerais*. Cada botão invoca `SaxonFunctions_XQuery.xQueryToText()`, que compila e executa a query `.xql` correspondente através do motor Saxon HE, escreve o resultado num ficheiro `.txt` no diretório do projeto (`ranking_vida.txt`, `grupos_racas.txt`, `estatisticas.txt`), e apresenta o conteúdo gerado na `JTextArea` do separador.

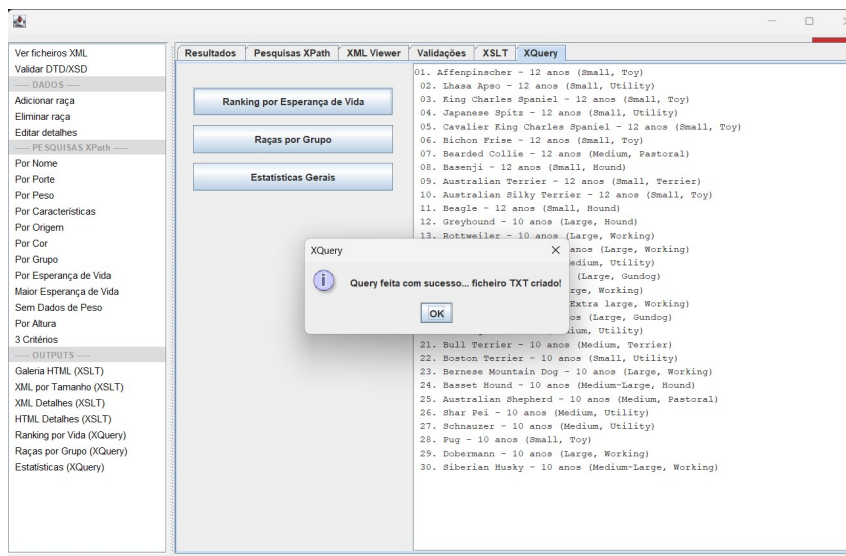


Figura 9.6: Janela XQuery — vista geral da interface.

9.7 Feedback ao Utilizador

A aplicação utiliza de forma consistente `JOptionPane` como mecanismo de feedback modal para todas as operações que alteram dados ou produzem erros. Caixas de diálogo de informação confirmam a adição, remoção ou edição bem-sucedida de registos; caixas de erro apresentam mensagens descritivas em caso de falha (como raça não encontrada, ficheiro em falta ou erro de rede); caixas de aviso indicam situações intermédias, como a ausência de ficheiros de esquema para validação. Esta abordagem mantém a interface principal limpa e focada, reservando o feedback pontual para janelas modais que não interrompem o fluxo de trabalho do utilizador.

Capítulo 10

Resultados e Análise

Este capítulo apresenta uma análise dos dados efetivamente extraídos e integrados pela aplicação, avaliando a qualidade e a cobertura da informação obtida a partir das duas fontes — Royal Kennel Club e Wikipédia — para o conjunto de raças trabalhado ao longo do projeto.

10.1 Cobertura do Conjunto de Dados

O conjunto base de raças fornecido para o trabalho é composto por **28 raças**: Af-fenpinscher, Greyhound, Rottweiler, Alaskan Malamute, Chow Chow, Lhasa Apso, King Charles Spaniel, Japanese Spitz, Irish Setter, Hovawart, Great Dane, English Setter, Cavalier King Charles Spaniel, Bulldog, Bull Terrier, Boston Terrier, Bichon Frise, Bernese Mountain Dog, Bearded Collie, Basset Hound, Basenji, Australian Terrier, Australian Shepherd, Australian Silky Terrier, Beagle, Shar Pei, Schnauzer e Pug. Todas as raças desta lista foram encontradas no Royal Kennel Club e integradas com sucesso na base de dados local.

A extração a partir do **Royal Kennel Club** revelou-se robusta para todos os campos principais: grupo, porte, nível de exercício, cuidados de pêlo, comprimento do pêlo e esperança de vida foram obtidos com sucesso para a totalidade das raças. Estes campos são apresentados de forma estruturada e consistente no site, o que facilitou a extração com JSoup.

A extração a partir da **Wikipédia**, por expressões regulares, apresentou maior variabilidade. Os campos de origem e cor foram obtidos com sucesso para a maioria das raças. Os dados biométricos — peso e altura — revelaram-se os mais inconsistentes: algumas páginas da Wikipédia não fazem distinção entre macho e fêmea (sendo usados os métodos *geral*), outras apresentam as medidas em unidades imperiais (libras e polegadas), que são convertidas automaticamente para o sistema métrico pelo método `parseMedida`. Raças menos populares ou com páginas da Wikipédia menos completas resultaram em campos em branco para peso e/ou altura, sendo esses registos identificáveis através da pesquisa XPath *"Sem Dados de Peso"*.

10.2 Análise por Grupo

A distribuição das 28 raças pelos grupos do Royal Kennel Club reflete a diversidade do conjunto selecionado. O grupo **Toy** é o mais representado, incluindo raças como o Affenpinscher, o Pug, o Bichon Frisé, o King Charles Spaniel, o Cavalier King Charles Spaniel, o Japanese Spitz e o Australian Silky Terrier — raças de pequeno porte tradicionalmente associadas à companhia. O grupo **Utility** reúne raças de propósito variado como o Chow Chow, o Lhasa Apso, o Bulldog, o Boston Terrier, o Shar Pei e o Schnauzer. Os grupos **Working** e **Hound** incluem raças de maior porte como o Rottweiler, o Alaskan Malamute, o Great Dane, o Greyhound, o Basset Hound, o Basenji e o Beagle. Os grupos **Gundog**, **Pastoral** e **Terrier** completam a distribuição com o Irish Setter, o English Setter, o Australian Shepherd, o Bearded Collie, o Bull Terrier e o Australian Terrier.

10.3 Análise por Porte e Esperança de Vida

O conjunto de raças cobre todo o espectro de portes disponíveis no Royal Kennel Club: *Small* (ex: Affenpinscher, Pug, Beagle), *Medium* (ex: Bulldog, Shar Pei, Bearded Collie) e *Large* (ex: Rottweiler, Great Dane, Alaskan Malamute). Esta variedade permitiu validar de forma abrangente a pesquisa XPath "*Por Porte*", confirmando que os filtros funcionam corretamente para os três valores possíveis.

No que respeita à esperança de vida, verifica-se a tendência conhecida de que as raças de maior porte tendem a ter menor longevidade: o Great Dane, por exemplo, apresenta uma das menores esperanças de vida do conjunto (tipicamente 8 a 10 anos), enquanto raças pequenas como o Beagle, o Lhasa Apso ou o Miniature Schnauzer atingem frequentemente os 12 a 15 anos. A query XQuery *Ranking por Vida* permite visualizar esta distribuição de forma ordenada, sendo um dos outputs mais informativos da aplicação.

10.4 Testes com Raças Adicionais

Para além do conjunto base fornecido, foram realizados testes adicionais com duas raças não incluídas na lista original: o **Doberman** (também conhecido como Dobermann) e o **Husky Siberiano** (*Siberian Husky*). O objetivo foi verificar o comportamento da aplicação com raças populares e bem documentadas, validando a generalidade do sistema de extração e integração.

10.4.1 Doberman

O Doberman pertence ao grupo **Working** do Royal Kennel Club, com porte *Large*. A extração dos dados do Royal Kennel Club decorreu sem problemas, obtendo-se todos os campos estruturados (grupo, porte, exercício, grooming, comprimento do pêlo e esperança de vida). A página da Wikipédia do Dobermann é detalhada e bem estruturada, pelo que os campos de origem (Alemanha), cor (preto e castanho, ou castanho e ferrugem) e dados biométricos foram extraídos corretamente pelas

expressões regulares. O Doberman apresenta uma esperança de vida de aproximadamente 10 a 13 anos e um peso típico de 34 a 45 kg nos machos.

10.4.2 Husky Siberiano

O Husky Siberiano pertence ao grupo **Working**, com porte *Medium*. A extração do Royal Kennel Club foi igualmente bem-sucedida. A Wikipédia apresenta a raça com dados biométricos detalhados e distinção entre macho e fêmea, o que permitiu testar os métodos `obtem_pesoMacho/obtem_pesoFemea` e `obtem_alturaMacho/obtem_alturaFemea` de forma independente — ao contrário de algumas raças do conjunto base que apenas apresentam valores gerais. A origem reportada é a Sibéria (Rússia), e as cores incluem uma grande variedade (preto, cinzento, vermelho e branco), o que permitiu validar a pesquisa XPath *"Por Cor"* com um valor multi-palavra.

10.4.3 Observações sobre os Testes Adicionais

A adição das duas raças extras decorreu sem necessitar de qualquer alteração ao código de extração ou integração, o que confirma a robustez e a generalidade da arquitetura implementada. O sistema de verificação de duplicados implementado no método `criaEGuardaCao` impediu a re-inserção acidental das raças em execuções subsequentes. A lógica de atribuição automática de identificador numérico sequencial (`obtemProximoId`) garantiu que os IDs 29 e 30 foram atribuídos corretamente ao Doberman e ao Husky Siberiano, respetivamente, sem colisões com o conjunto base.